

```
module fulladd(ai, bi, ci, o, co);
    input ai, bi, ci;
    output o, co;
    wire s1;

    xor x1(s1, ai, bi), x2(o, s1, ci);

    assign co = (ai + bi + ci) >= 2;
endmodule
```

```
module testadd;
    reg [2:0] y;
    wire      o, co;
    reg       clk;

    fulladd a1(y[0], y[1], y[2], o, co);

    initial begin
        y = 0; clk = 0;
        $monitor($time, "%d%d%d",
            y[2], y[1], y[0], co, o);
    end

    always #10 clk = ~clk;
    always @(posedge clk) y <= y + 1;
endmodule
```

(a)

```
entity XOR2 is
    port (a: out Bit; b: in Bit);
end XOR2;
```

```
architecture arch1 of XOR2 is
begin
    A1: o <= (a xor b);
end arch1;
```

```
entity fulladd is
    port (ai, bi, ci: in Bit;
          o, co: out Bit);
end fulladd;
```

```
architecture arch1 of fulladd is
    signal s1 : Bit;
    component XOR2
        port (a: out Bit; b: in Bit);
    end component
    for X1, X2: XOR2 use entity Work.XOR2;
begin
    X1: XOR2 port map (s1, ai, bi);
    X2: XOR2 port map (o, s1, ci);
    A1: co <= (ai and bi) or (ai and ci)
        or (bi and ci);
end arch1;
```

(b)

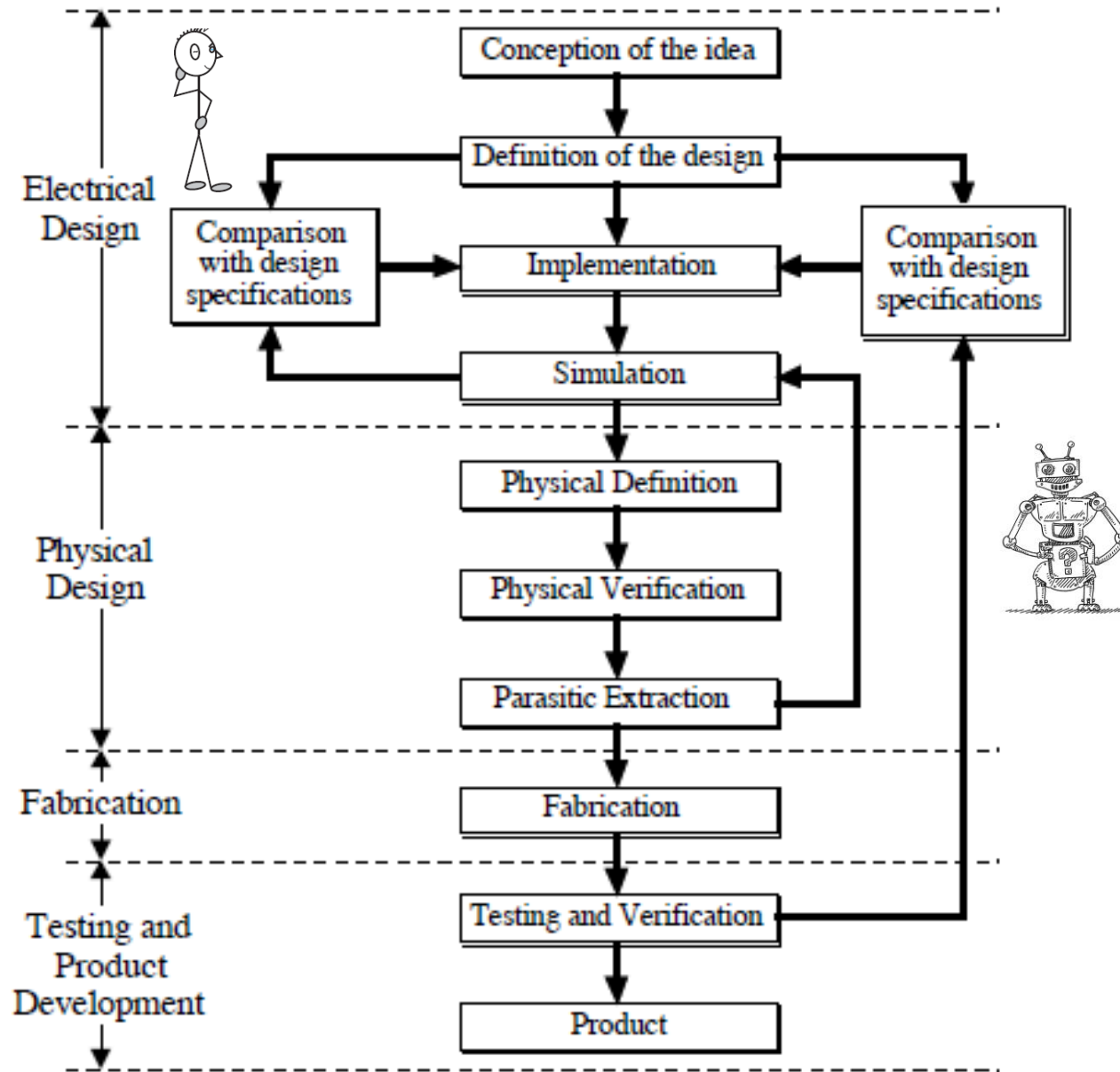
בדיקת מערכות ספרתיות ואימות תכנון

פרק 2

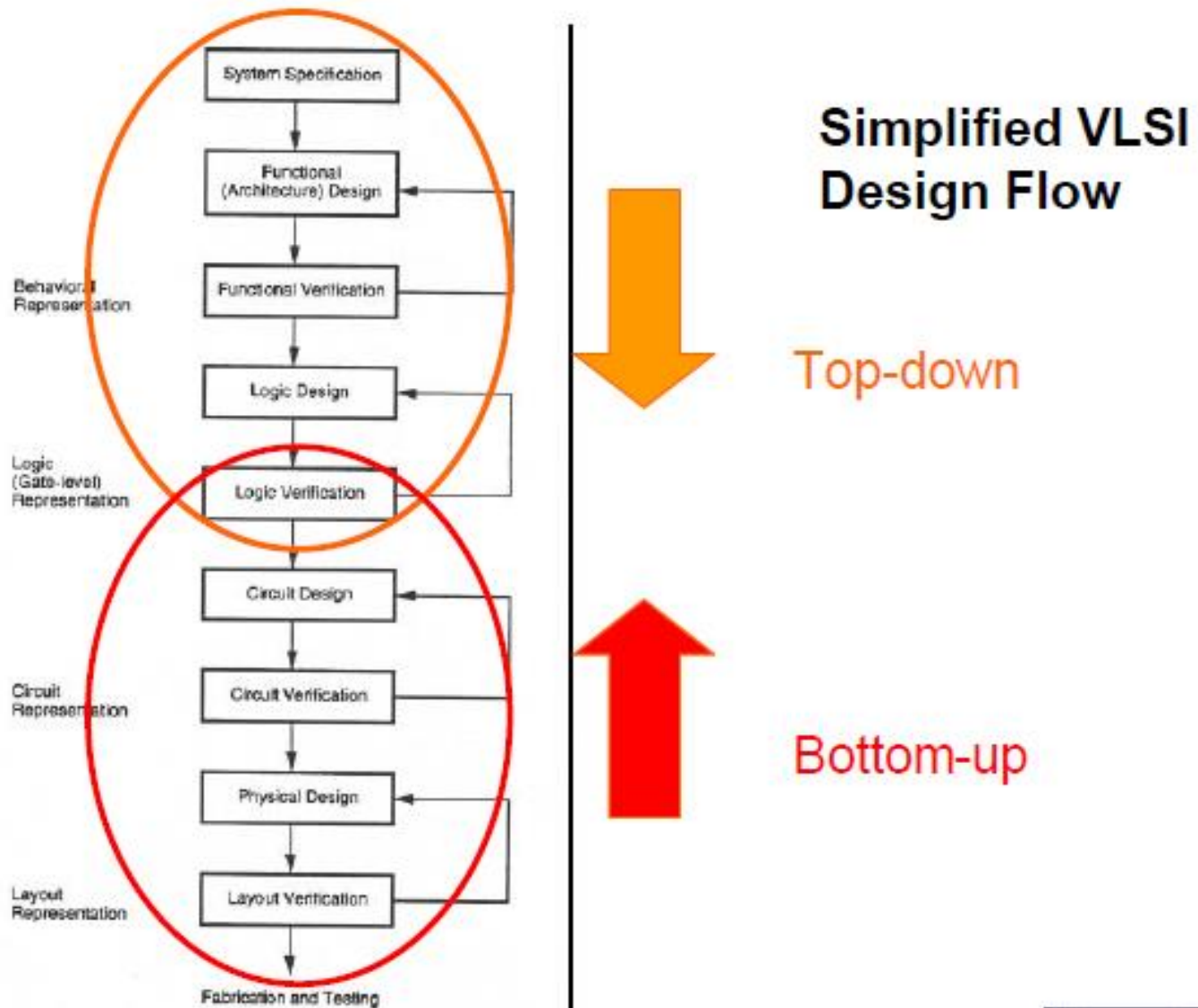
עקרונות בדיקות

בדיקות בשלבי כתיבת קוד

שלבי תכנון ובדיקות



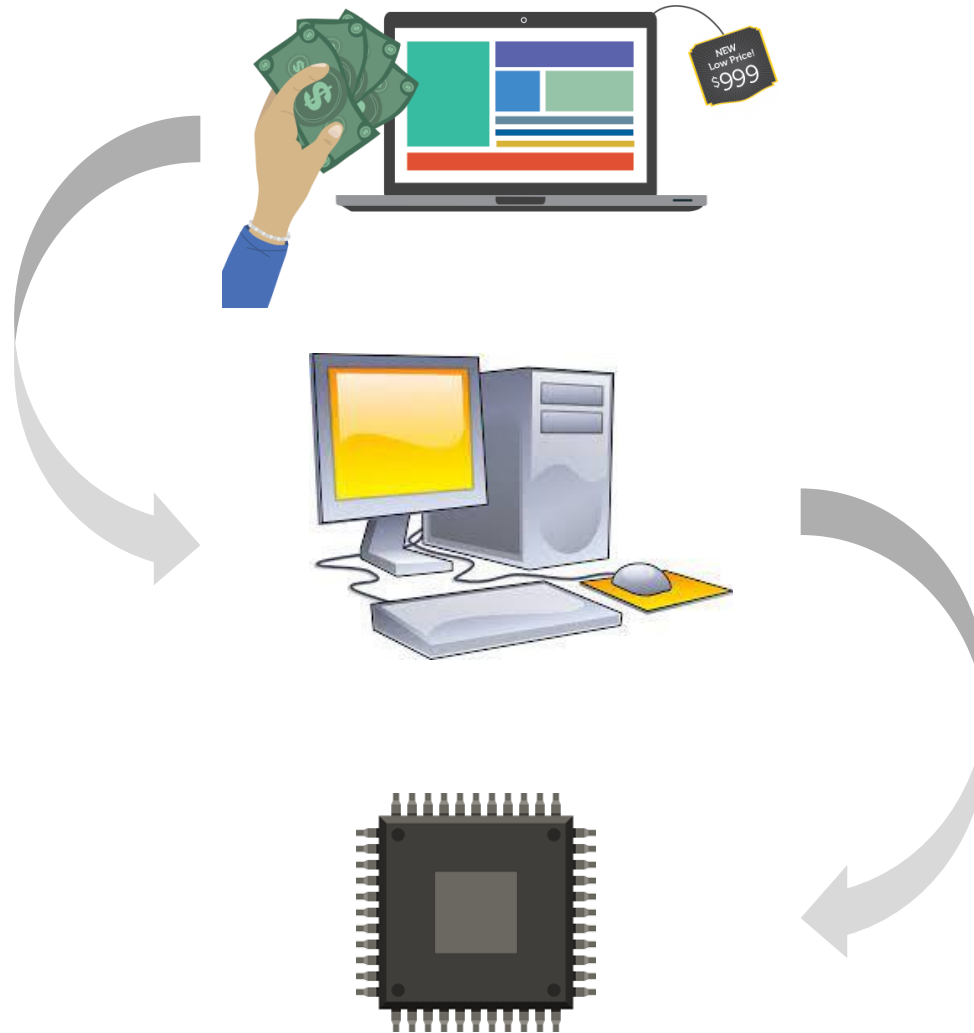
הגדרות דרישות לתכנון



אתגרי אימות

- איך אנחנו יודעים כי התכנון נכון?
- איך אנחנו יודעים כי המוצר מתנהג כצפוי?
- איך אנחנו יודעים מה יש לנו לבדוק ככל?
- כיצד אנו מתמודדים עם גודל וסיבוכיות של המוצרים?

דרישות שונות



הגדרת תכנון אלקטרוני



1. לבחור פתרון

- לבחון פתרונות קודמים
- לבחור פתרון פשוט

2. לחקור את הפתרון

- לנתח ביצועים (ללא מחשב)
- לקבוע את החוזקות והחולשות של הפתרון

שלבי תכנון אלקטרוני

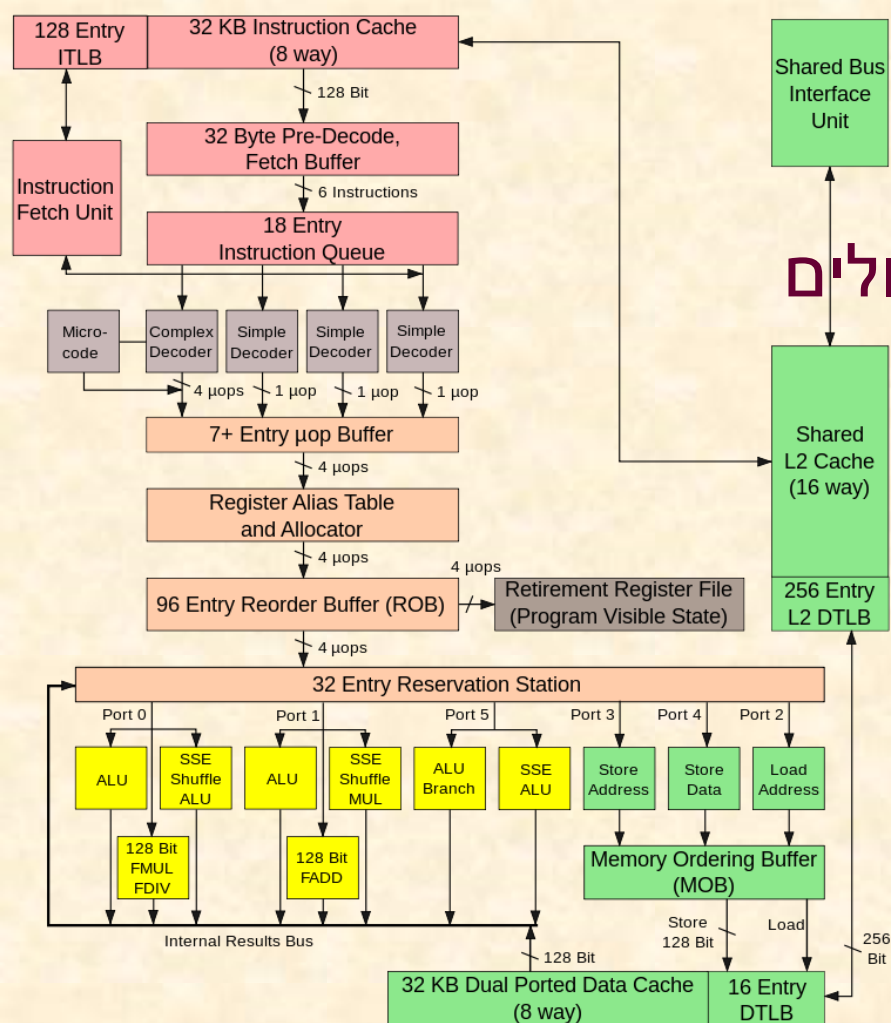
3. שינוי של הפתרון

- בחירת עקרונות מפתח, מושגים וטכניקות ליישום
- להעריך את השינויים באמצעות ניתוח (עדיין לא מחשבים)

4. אימות של הפתרון

- שימוש בסימולטור עם מודלים מדויקים לאמת את הפתרון
- חילוקי דעות גדולים עם ניתוח ביד: אימות המחשב יש לבחון היטב

עקרונות תכנון אלקטרוני



Intel Core 2 Architecture

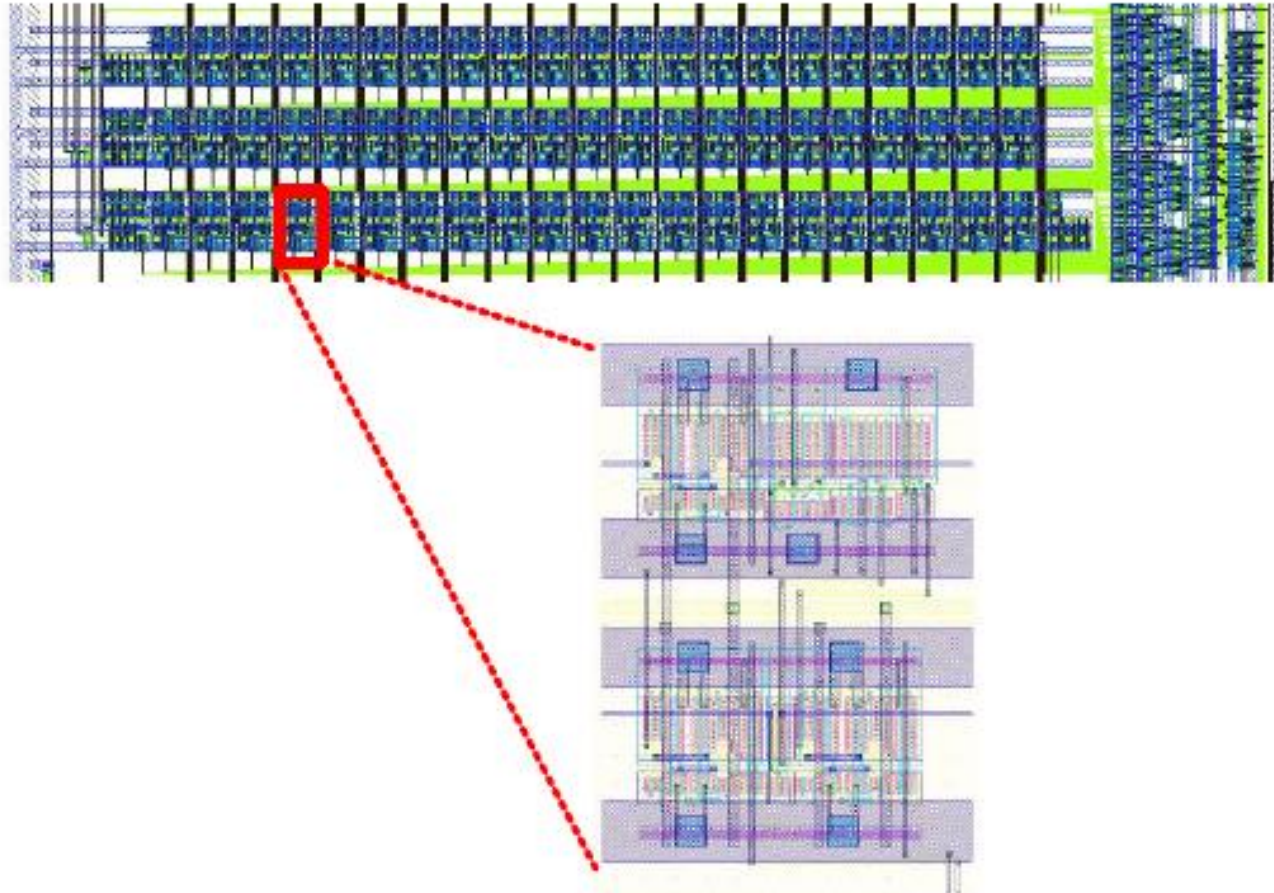
- **היררכיה: הטכניקה כוללת**
חלוקת מודול לתוך תת-מודולים
ואז חוזרים על פעולה זו על
תת-מודולים, עד
שהמורכבות של חלקים
קטנים יותר ניתנת לניהול.

עקרונות תכנון אלקטרוני (המשך)

- **סדירות**: פירוק היררכי של מערכת גדולה צריך לגרום לא רק לבלוקים פשוטים, אבל גם דומים, ככל האפשר. סדירות בדרך כלל מפחיתה את מספר המודולים השונים שנדרש לתכנן ולבדוק בכל הרמות של ההפשטה.

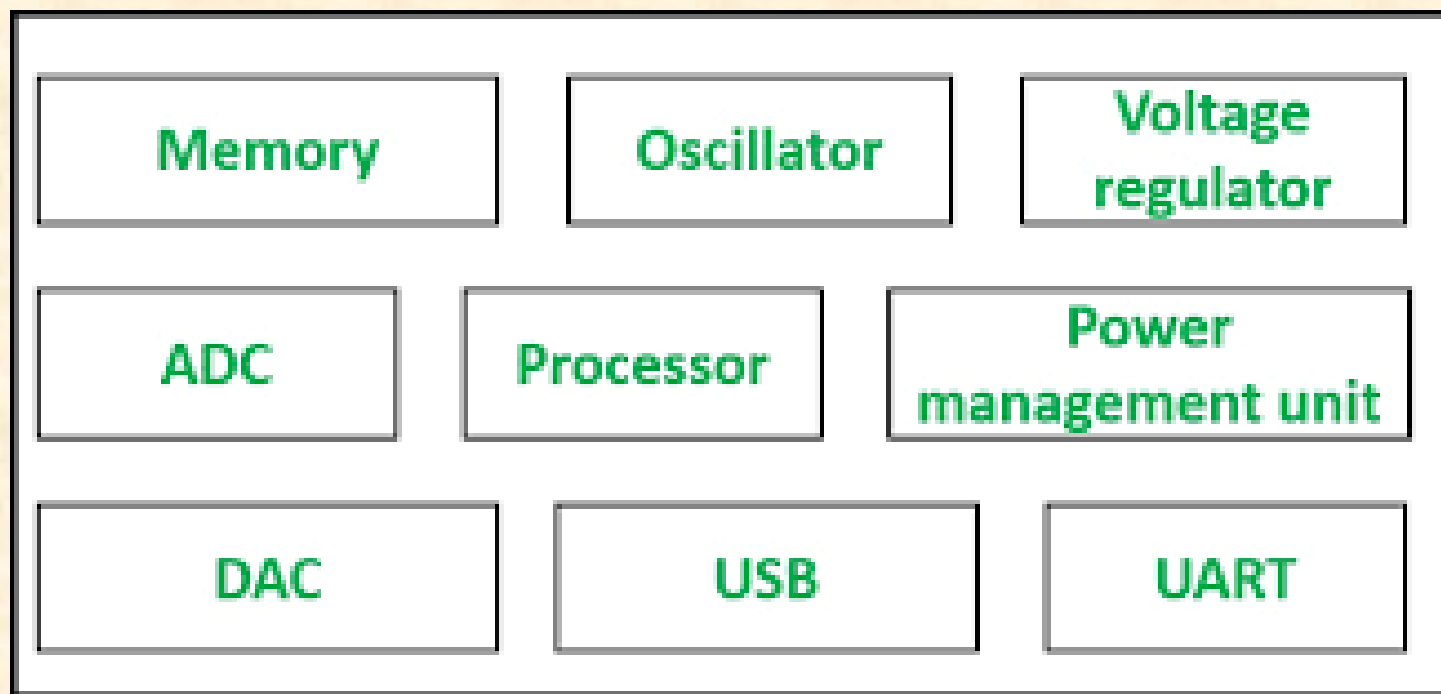
דוגמה לתכנון היררכי וסידורי

Example: 16-bit Binary Sorter



עקרונות תכנון אלקטרוני (המשך)

- מודולריות: בלוקים פונקציונליים שונים של המערכת חייבים להיות מוגדרים היטב מבחינת פונקציות והממשקים שלהם.

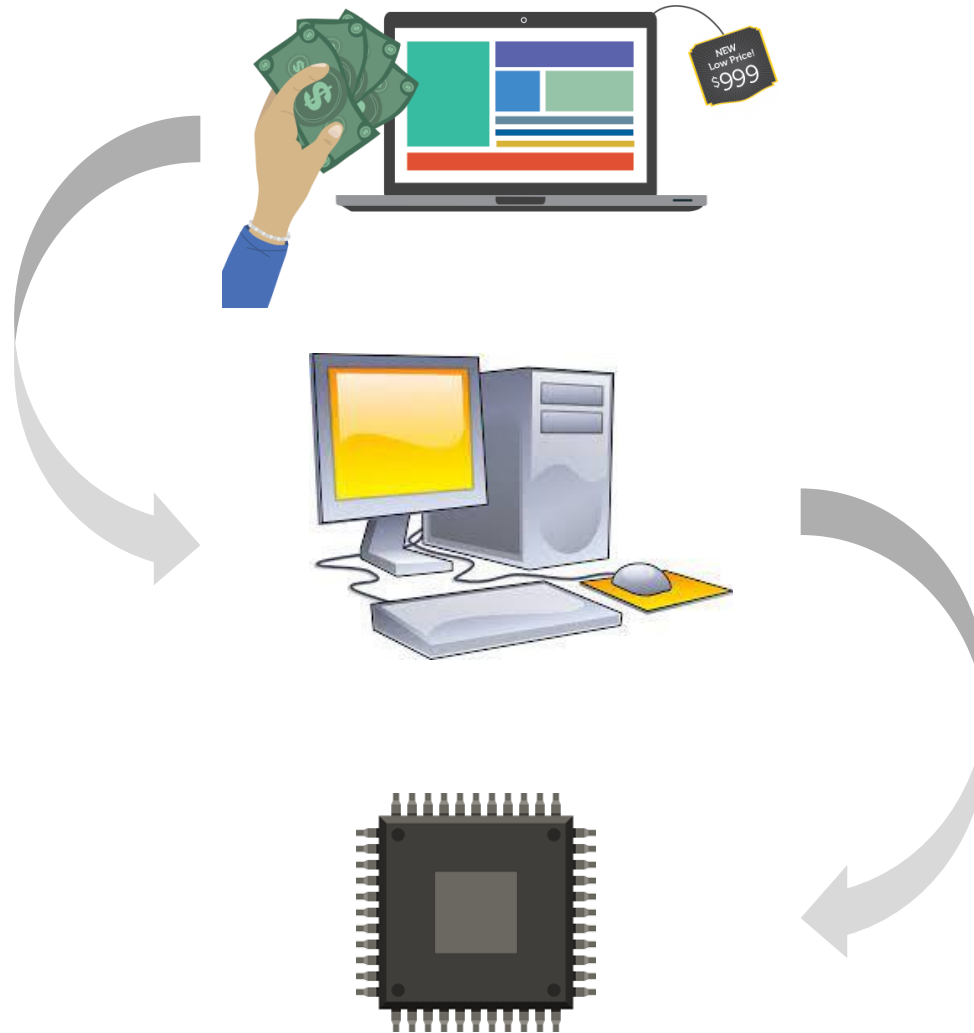


עקרונות תכנון אלקטרוני (המשך)

- לוקאליות: פרטים פנימיים נישארים ברמה המקומית. הרעיון של לוקאליות גם מבטיח קשרים בין המודולים השכנים.

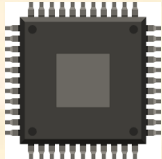


דרישות שונות (תזכורת)



שלושה המושגים הקשורים

1. Verification מודא שכל שלב התכנון עושה מה שהוא אמור לעשות



2. Validation מודא שהתוצאות הסופיות מספקות את הדרישות



3. Certification ערבות כתובה כי המוצר מקובל לשימוש



תוכנית בדיקה לאימות



- לוח זמנים
- דרישות לכלים
- קלט והקריטריונים לסיום
- בדיקות ספציפיות
- מה הציפיות מכל בדיקה
- מה לא מכוסה על ידי הבדיקות

אסטרטגיה אימות

- **בדיקת כל הממשקים (בנפרד ובשילובים)**

- ממשק מערכת: שעון, איפוס

- ממשקים on / off של כל האותות

- ממשקים של נתוני קלט; טווחים של ערכים, תת קבוצה של ערכים

- **בדיקת כל הפונקציונליות של הרכיב**

- לאפשר כל הפונקציות להיבדק בנפרד

- לאפשר כל הפונקציות להיבחן בקבוצות

אסטרטגיה אימות (המשך)

- **בדיקת כל המקרים בפינות**

- במקרים המפורטים על ידי מהנדס במפרט

- דברים שדפקו את התכנון בפעם האחרונה

- **כמו כן לבדוק את**

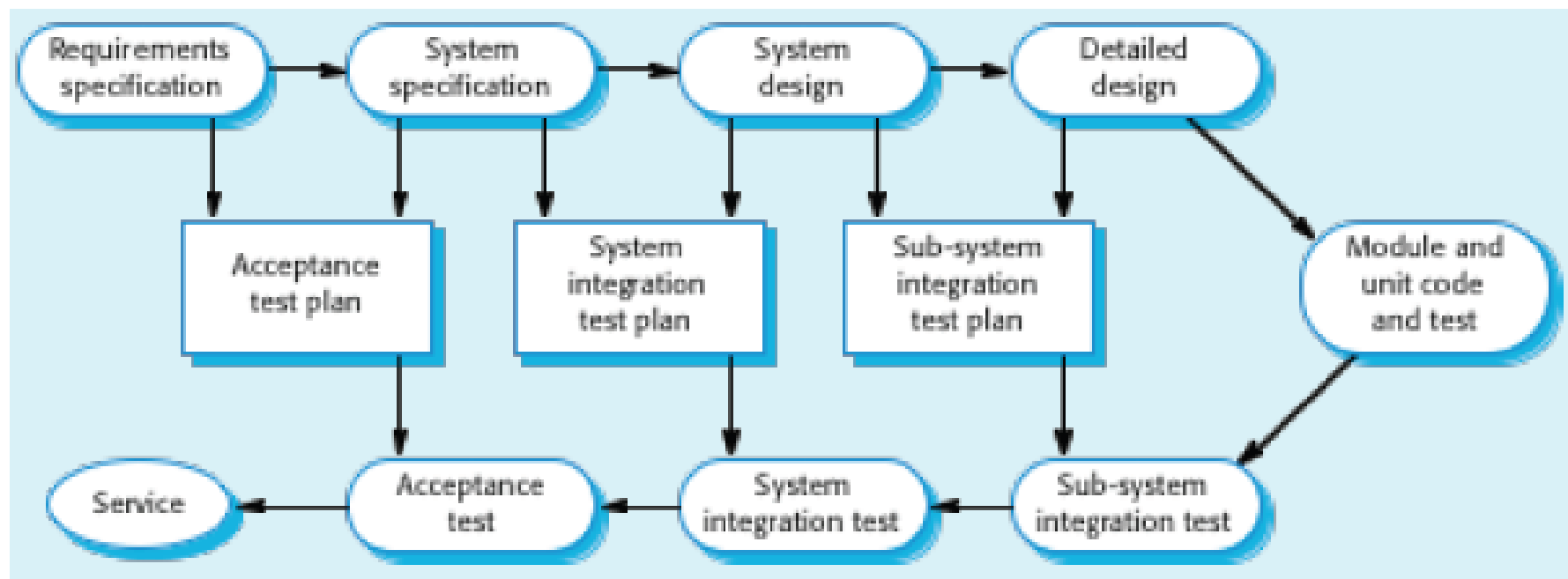
- ממשקים, פונקציונליות, שגיאות להיות משולבות באופן

- אקראי

- שילוב עם חוסר העבודה המעשית ואותות חשמליות לא

- חוקיות

תוכנית בדיקה לאימות (המשך)



רמות הבדיקה

- **Unit test** בדיקת יחידה - בדיקת חתיכות קטנות של תכנון, נעשה על ידי מהנדס חשמל, ושאר הבדיקות על ידי מהנדס בדיקות
- **Module/functional test** בדיקת מודול / מבחן פונקציונלי - בדיקות ברמת ה-API נעשה על ידי קבוצת בדיקות
- **Integration test** בדיקת אינטגרציה - בדיקות שחתיכות בדיקות עובדות יחד, נעשה על ידי קבוצת בדיקות
- **Acceptance test** מבחן קבלה - בדיקות הרכיב כולו, נעשה על ידי הלקוח (או תחליף)
- **בדיקת בטא β** - לתת לכמה לקוחות להשתמש במוצר לפני ייצור מלא



מה זה

- התכנון אינו תואם את מפרט

➤ בעיה אחת: המפרט השלם והעקבי אינו קיים למוצרים רבים

- משהו שהלקוח יהיה להתלונן על

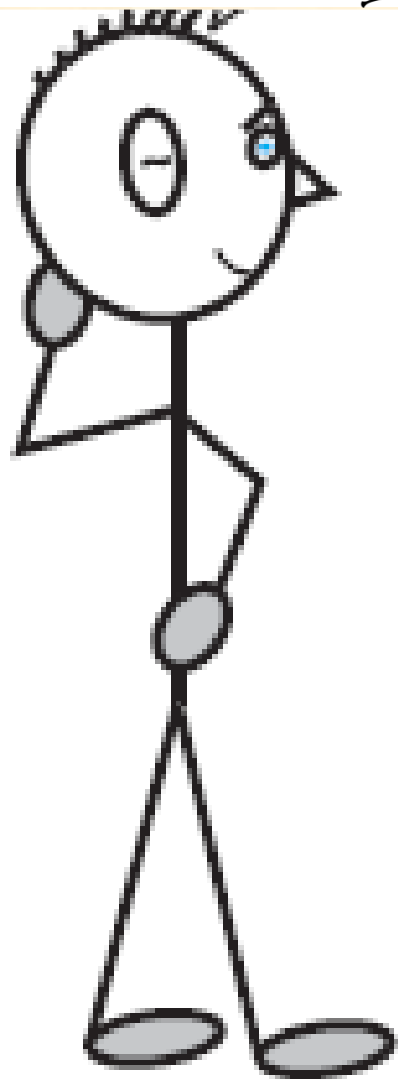
➤ "זה לא באג, זה תכונה"



“It’s not a bug, it’s a feature”



מה המקורות של

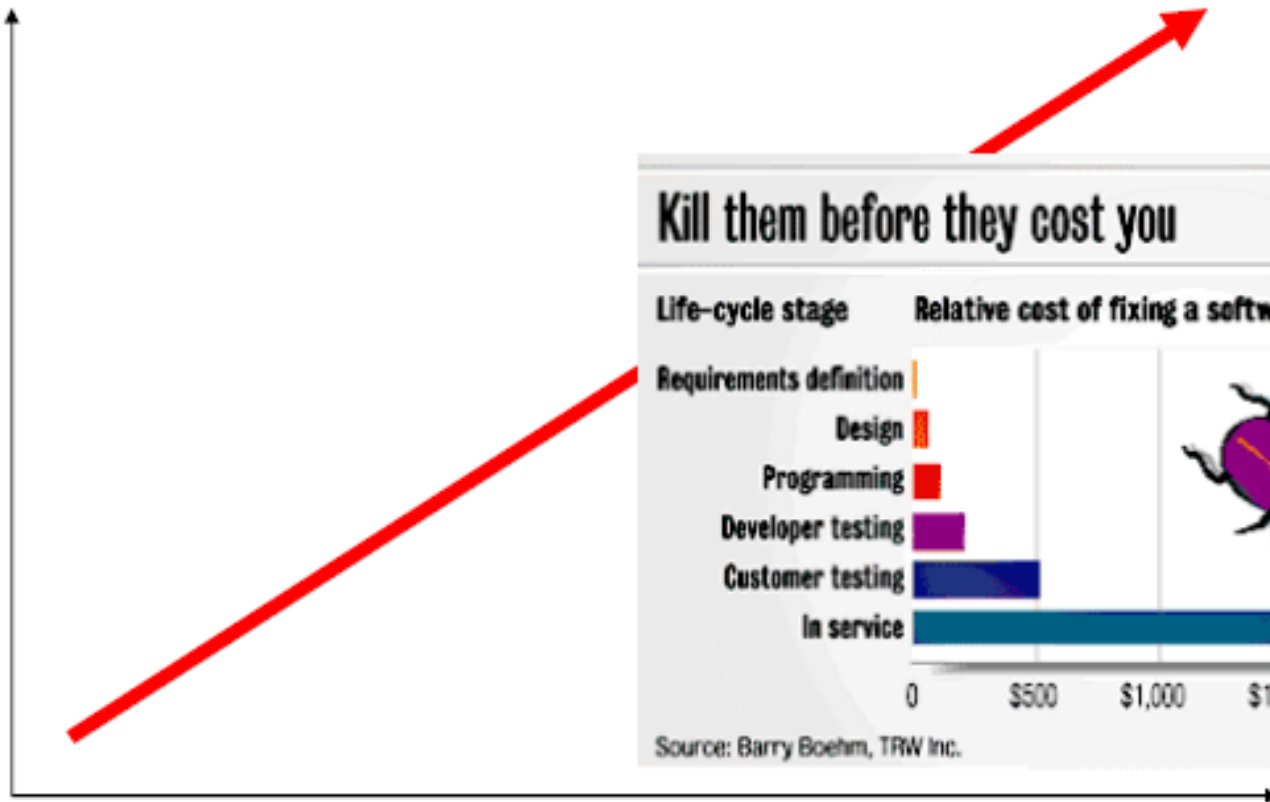


Type of Bug	%
"Goof"	12.7
Miscommunication	11.4
Microarchitecture	9.3
Logic/Microcode Changes	9.3
Corner Cases	8.0
Power Down	5.7
Documentation	4.4
Complexity	3.9
Initialization	3.4
Incorrect RTL Assertions	2.8
Design Mistake	2.6



מה המחיר מצאית

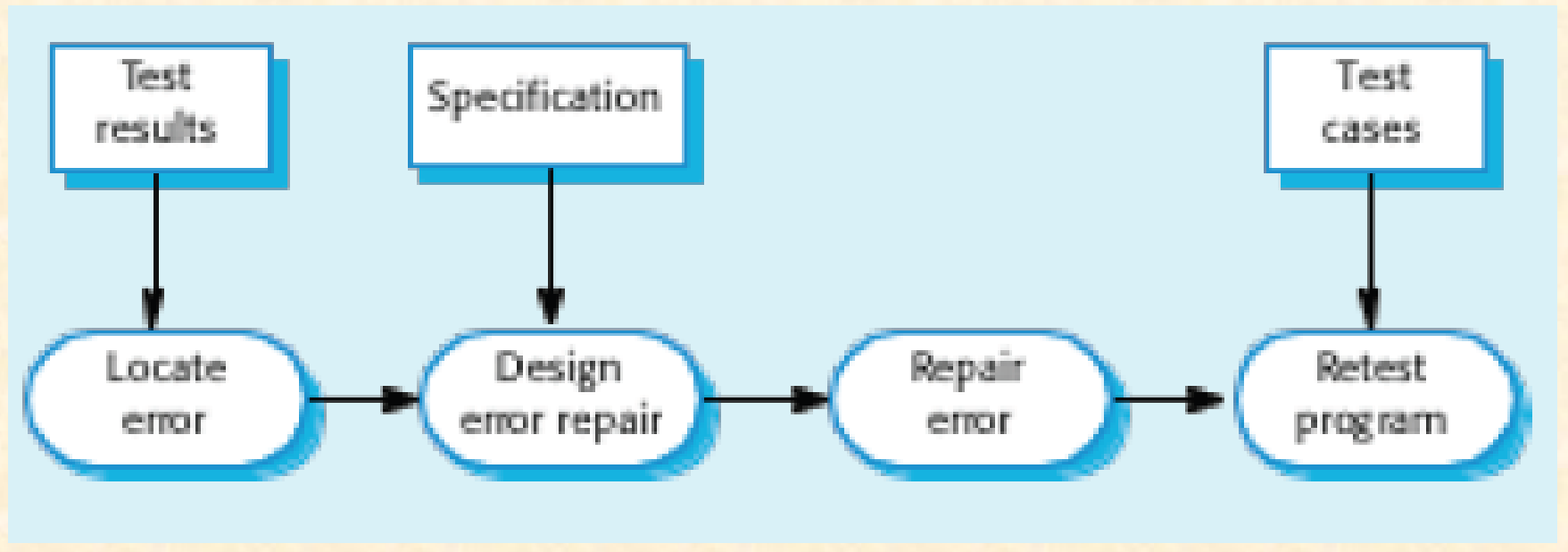
$\log(\text{cost})$



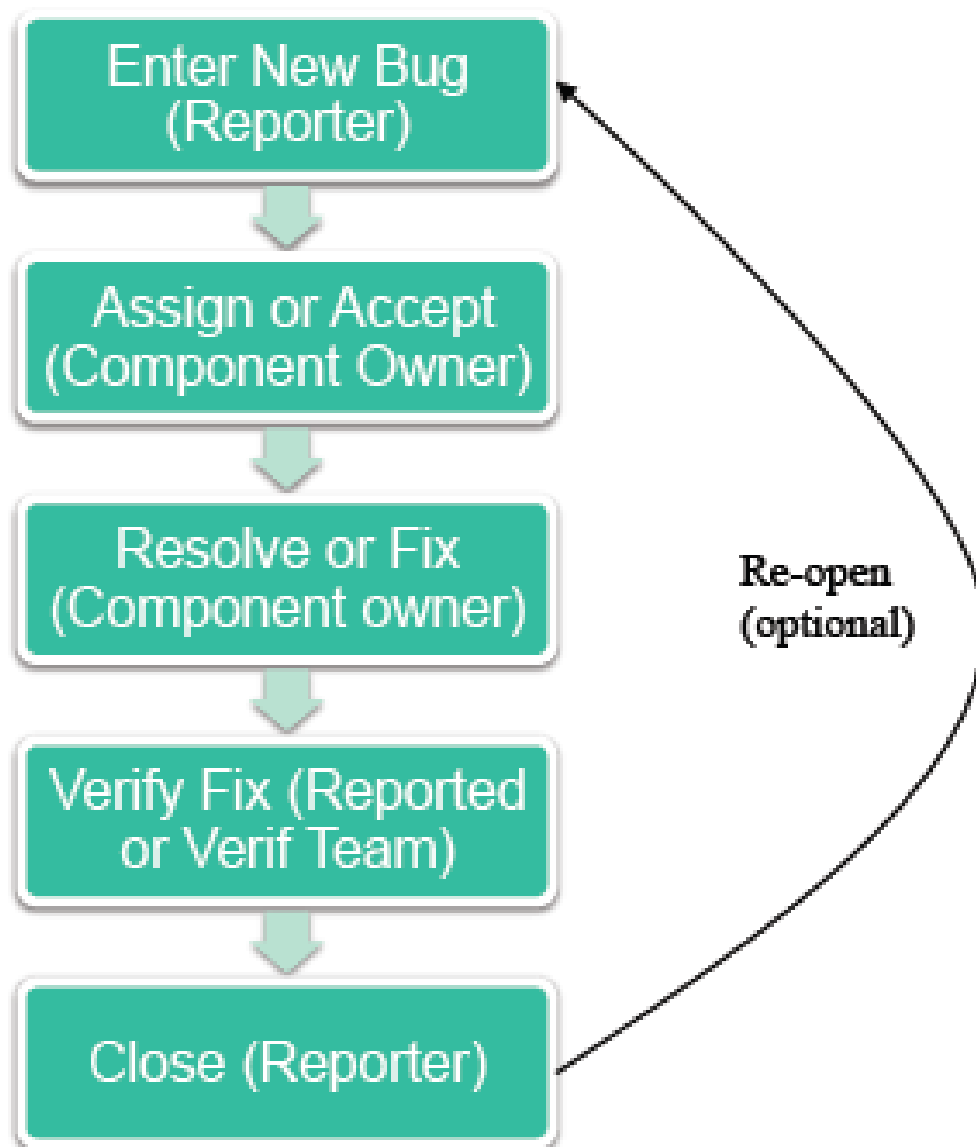
Product Lifecycle Phase

איתור באגים

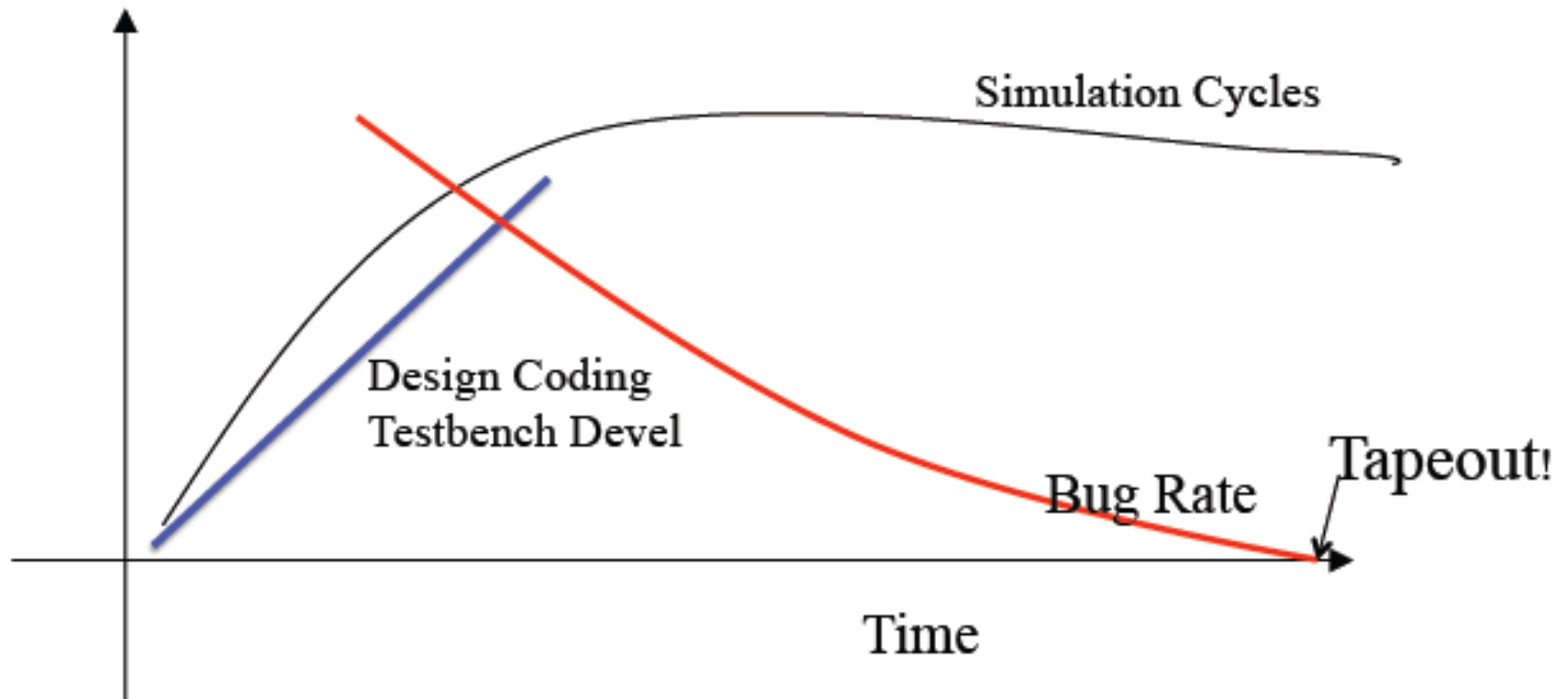
- עוסק באיתור ותיקון שגיאות
- כרוך בניסוח השערה על התנהגות התכנון: אזי בודקים את ההשערות האלה כדי למצוא את שגיאות התכנון



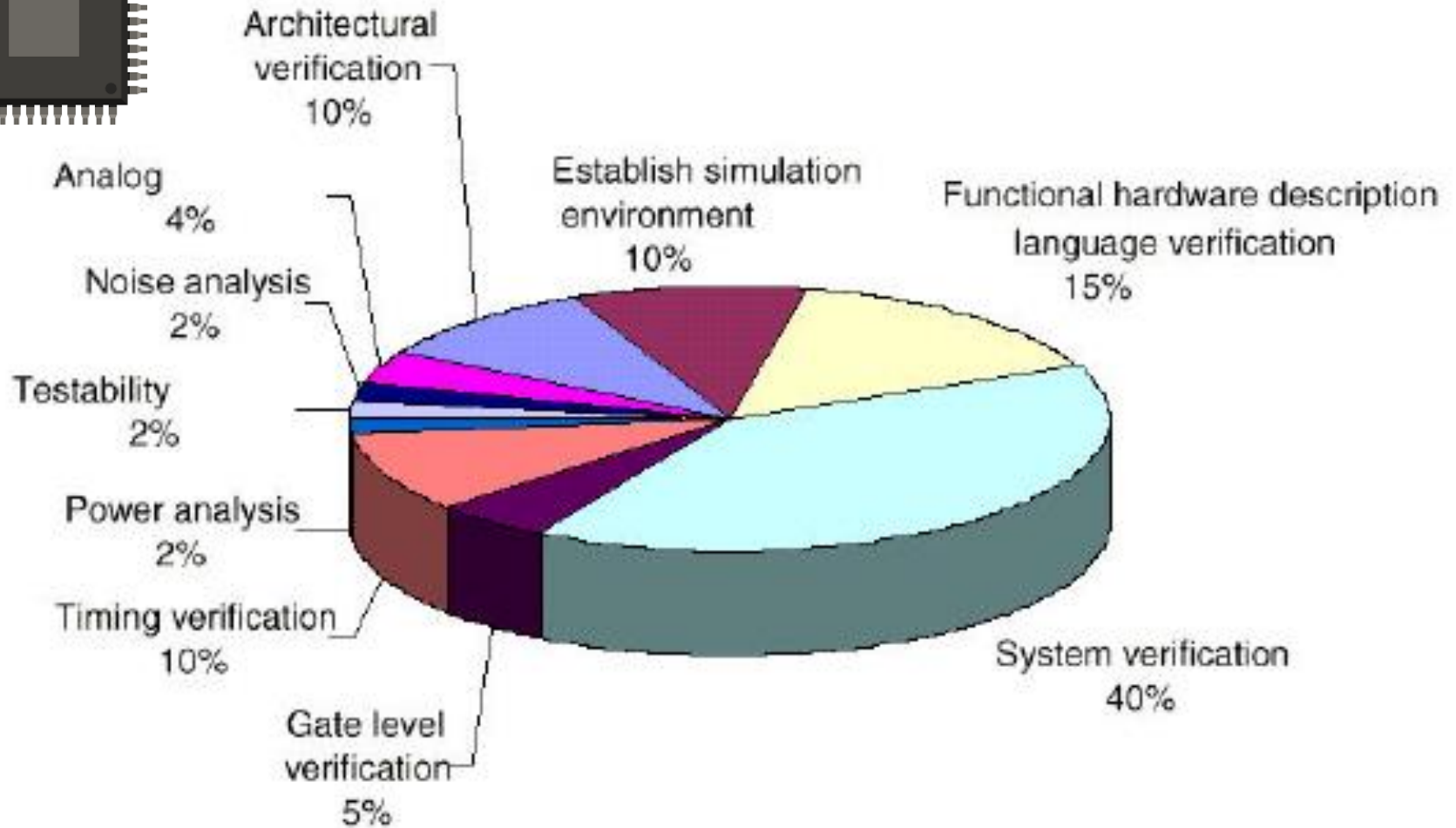
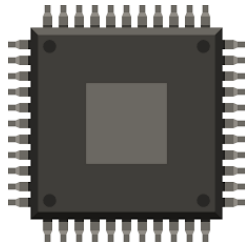
חיים של הבאג



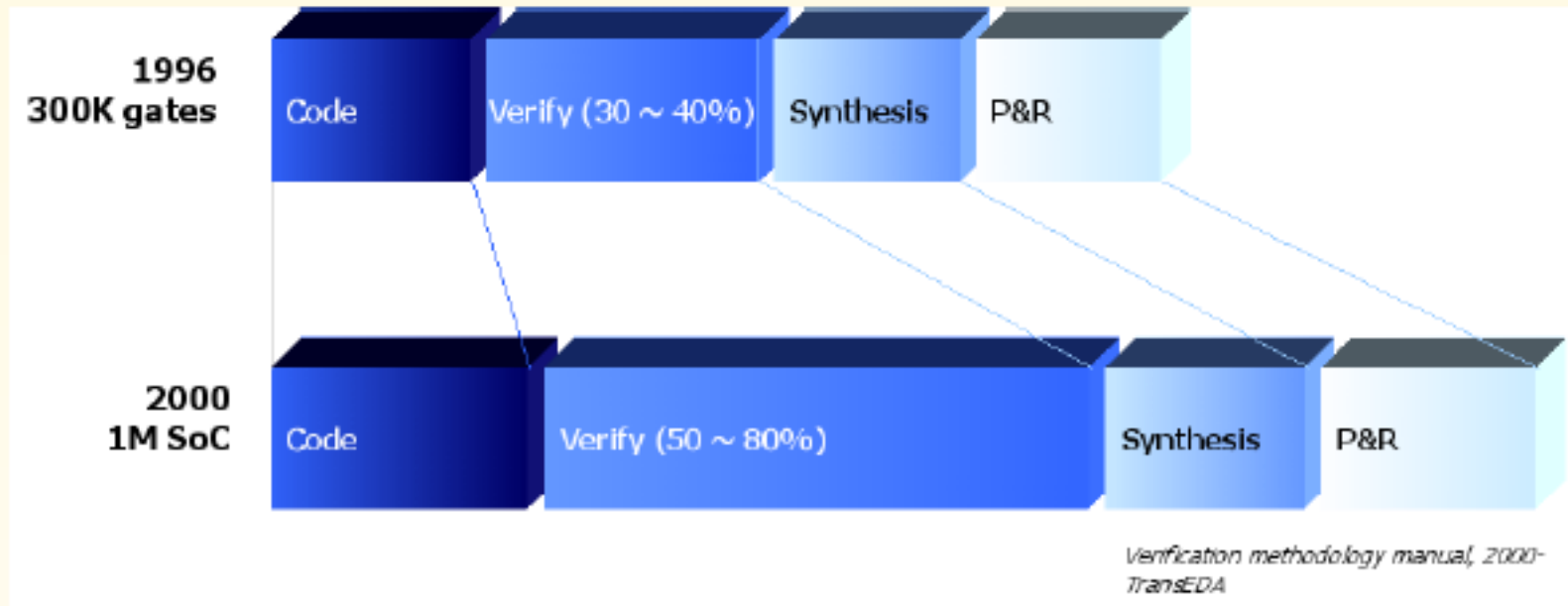
קצב מציאת באגים לאורך זמן הפרואקט



פילוג של בדיקות שונות



מאמץ תכנון ובדיקות



Verification is becoming an increasing part of the design effort

גישות כלליות לאימות

• אימות

- להריץ את הרכיב כדי לקבוע את ההתנהגות שלו
- להזריק תקלות מכוונות כגי לקבוע את תגובות הרכיב

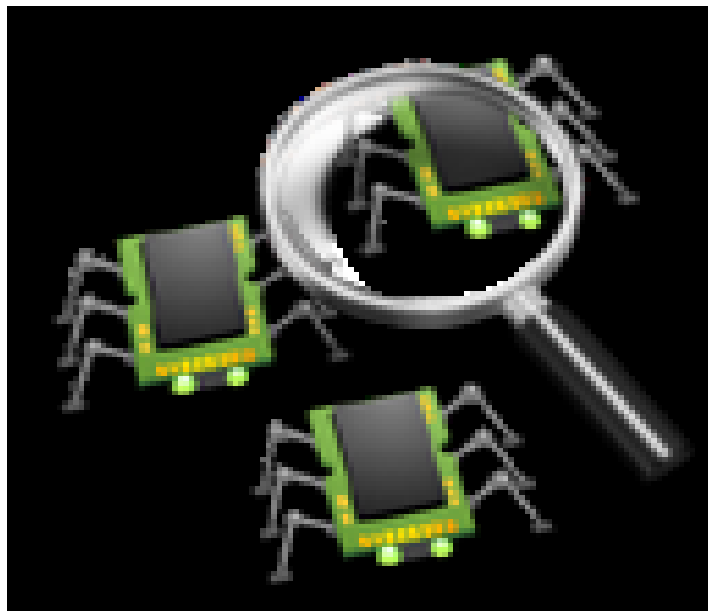
■ אנליזה

- לקבוע האם תכונות רצויות נכונות, האם קיימים מאפיינים רצויים
- למצוא סתירות בין שלבי התכנון
- לקבוע האם כללי התכנון התקיימו
- לבחון מסמכים וביקורות

מהי האימות?

- אימות אינו טסט או סדרה של טסטים
- אימות הוא תהליך המשמש כדי להוכיח את הנכונות התפקודית של התכנון. זהו מעשה להבטיח כי התכנון הלוגי תואם את מפרט.
- האימות אינו להבטיח כי המפרט הוא הנכון.

אנליזה - Analysis



- **בדיקת תכנון & תיעוד**

- אין ריצה בפועל של תכנון אמיתי

- יעיל מאוד במציאת פגמים

לדרישות התכנון

- **בחירת רמות משתנות של כלים / התערבות אנושית**

- נע בין ניתוח סטטי מלא על ידי כלים שונים...

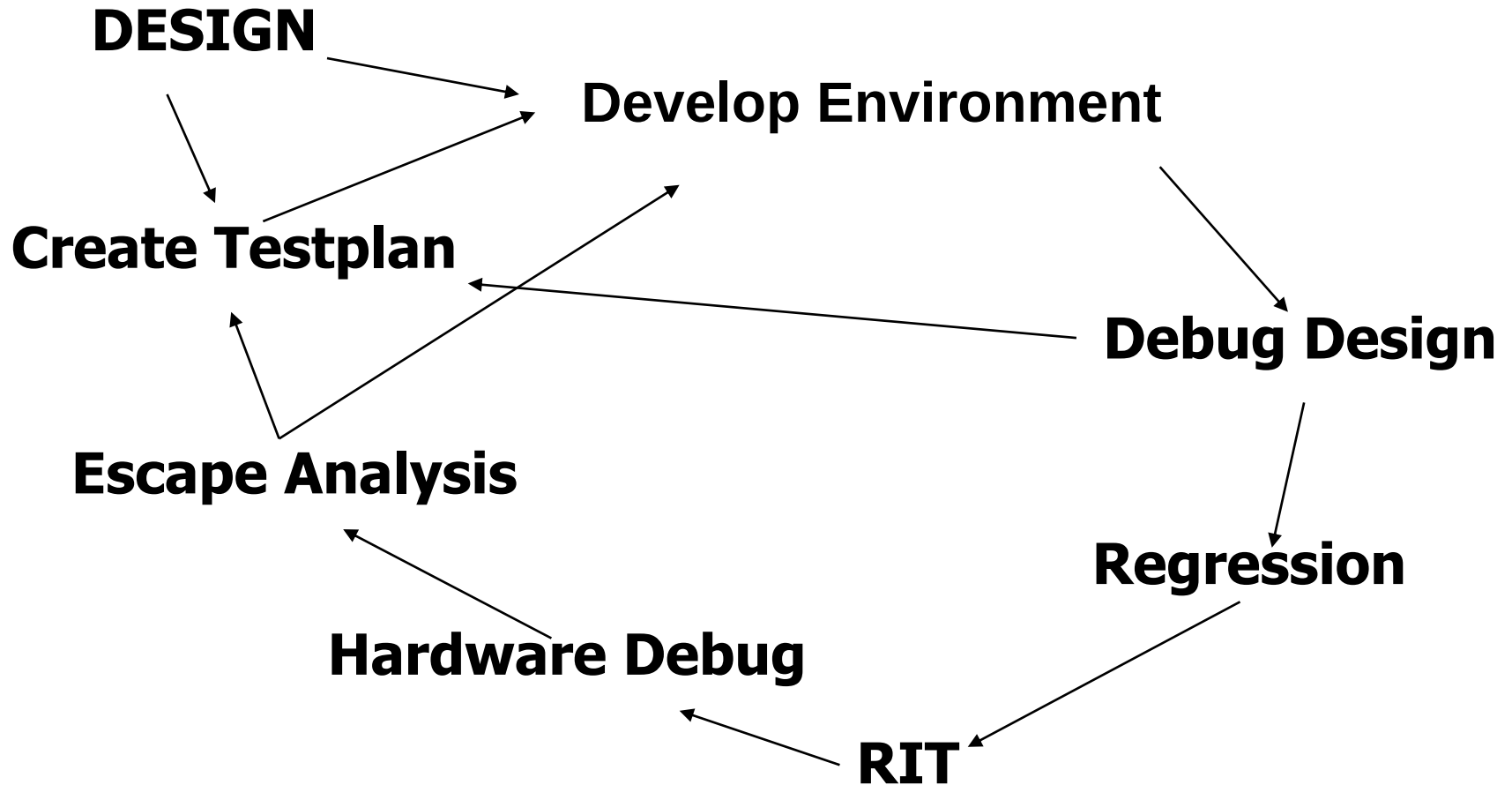
- לבני אדם שיושבים בחדר ישיבות ומסתכלים במסמכים

של הדרישות

תהליך בדיקות



מחזור אימות



Test Coverage – כיסוי טסטים

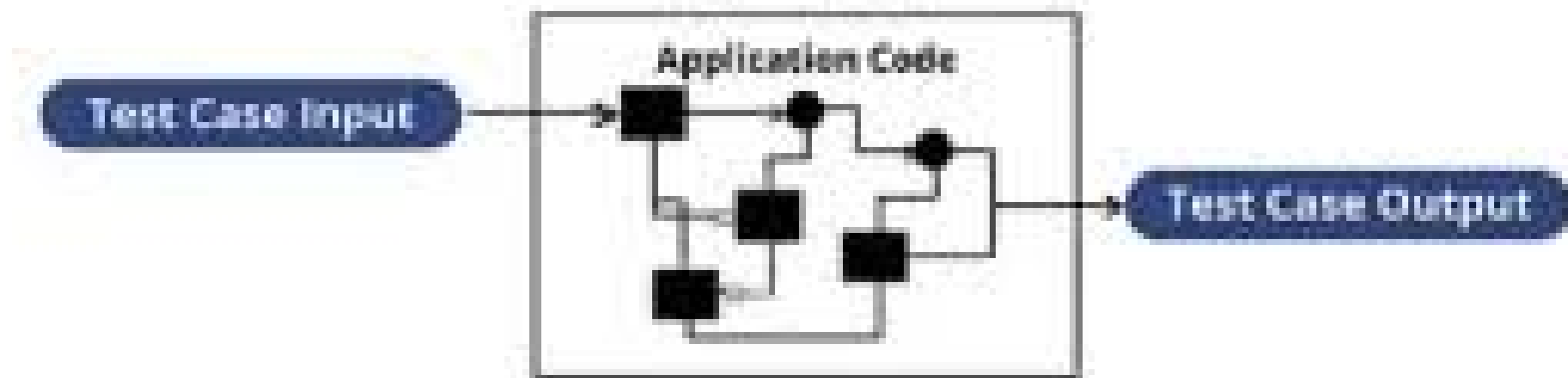


- **כיסוי דרישות:**
 - כל הדרישות נבדקו
 - בכל ההפעלות העיקריות
 - **כיסוי תרחיש**
 - כל הרצפים האפשריים **נבדקו**
 - **כיסוי Statechart**
 - כל המצבים נבדקו
 - **כיסוי קוד**
 - כל שורה בתכנון הורץ
- (100% כיסוי של מעברים מותניים)

בדיקות שונות

- White-box (בדיקה מבנית)
- תסתכל על מבנה התכנון ובדיקות התכנון

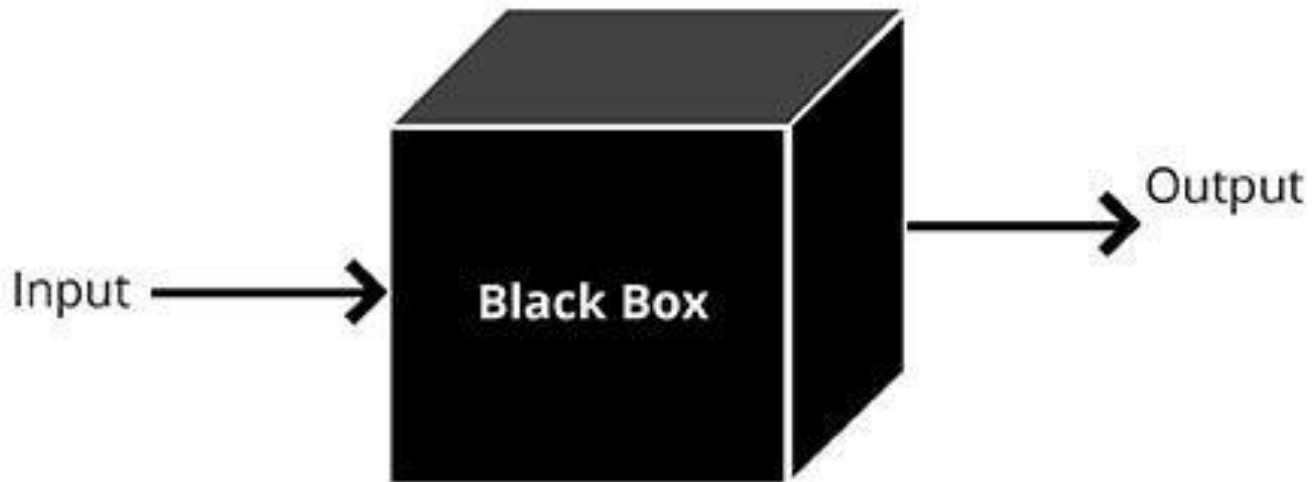
WHITE BOX TESTING APPROACH



בדיקות שונות (המשך)

- **Black-box (בדיקה פונקציונלית)**
 - בדיקת כל פריט מול המפרט הפונקציונלי
 - כמו כן, מבחן עמידות לטיפול / שגיאה

BLACK BOX TESTING APPROACH



מה זה HDL?

Hardware Description Language (HDL)

שפת תכנות שיכולה למדל, לייצר ולסמלץ תכנון דיגיטלי
מטפלת ב-

➤ חומרה מקביליות

➤ תכנון מקבילי

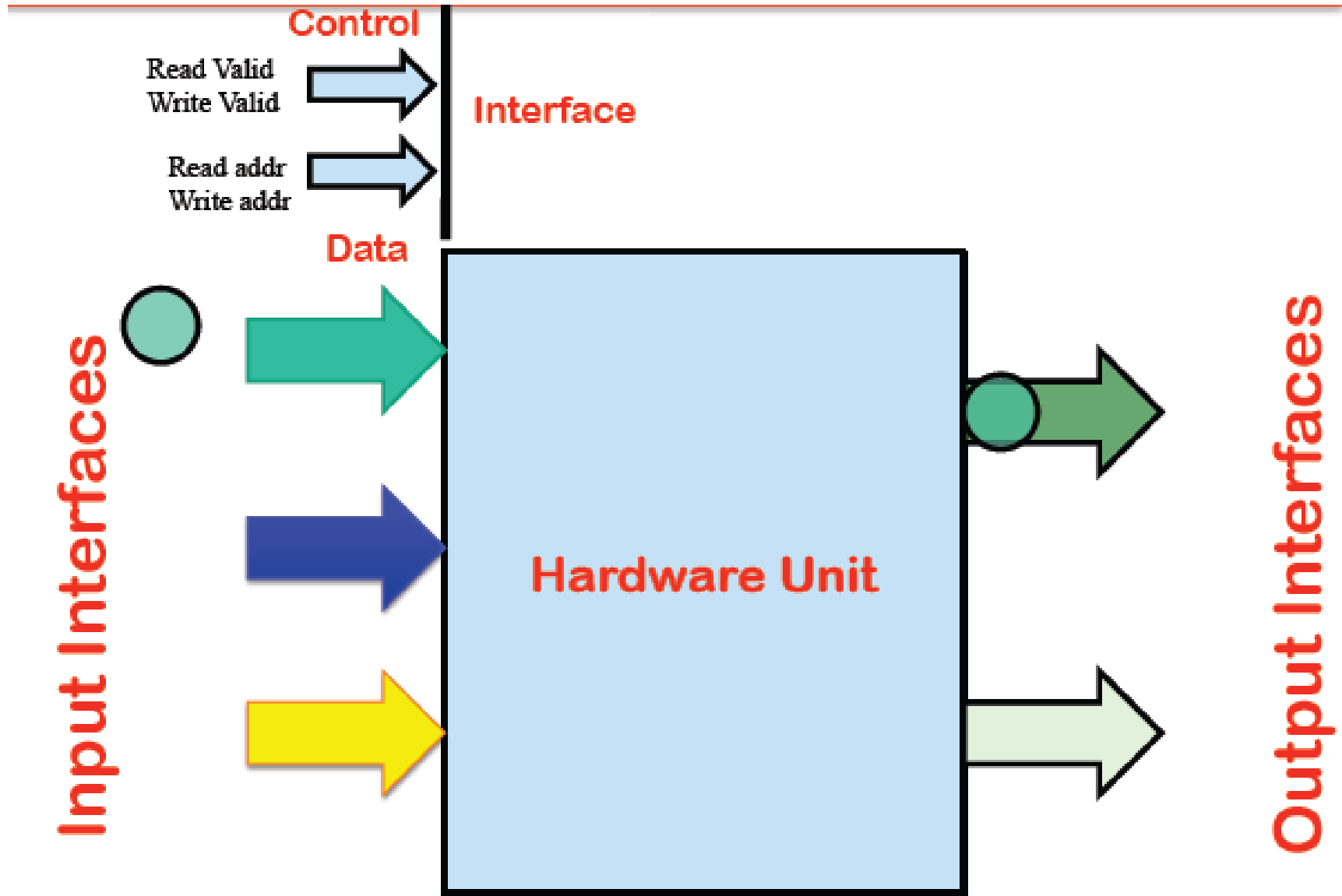
➤ סמנטיקה עבור אותות הזמן

The logo for Verilog HDL, featuring the word "Verilog" in a large, white, serif font and "HDL" in a smaller, bold, white, sans-serif font, both set against a dark blue rectangular background.

Testbenches – סביבת בדיקות

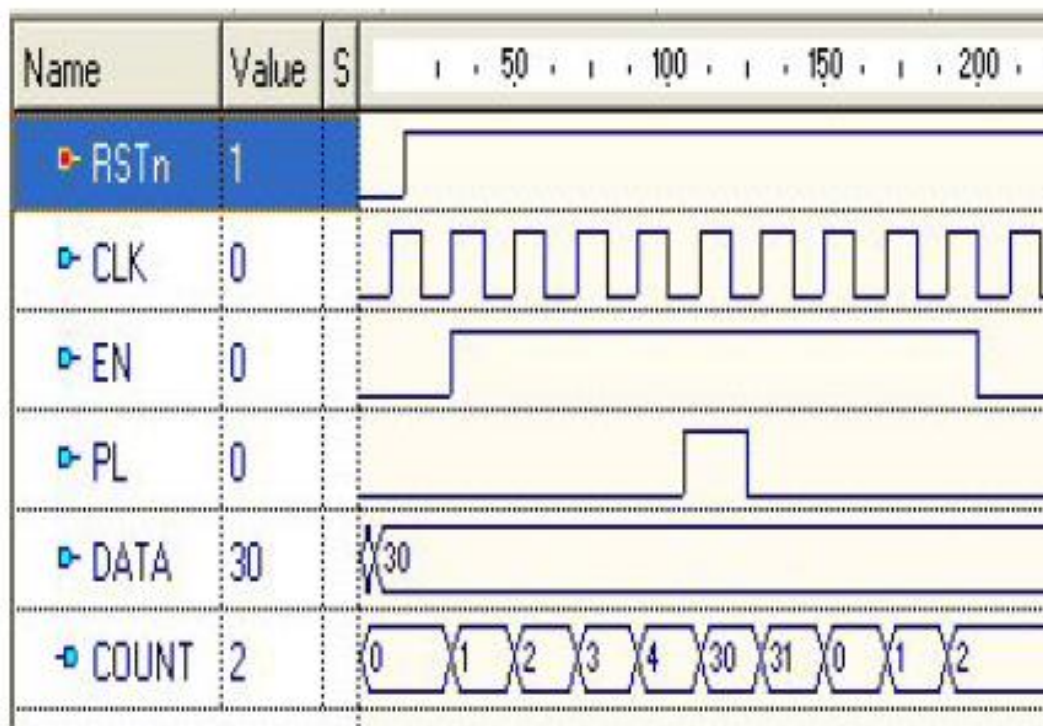
- טסטים ב-VHDL ו-Verilog מתייחסים לקוד שמשמש ליצירת רצף קלט לתכנון הנבדק (DUT) ובאופן אופציונאלי לבדיקת התוצאות.
- טסטי אימות מספקים את הפלט ובודקים את התוצאות.
- האתגר של האימות הוא לקבוע מה הקלט ומה הפלט הצפוי של תכנון תקין

אימות יחידת החומרה

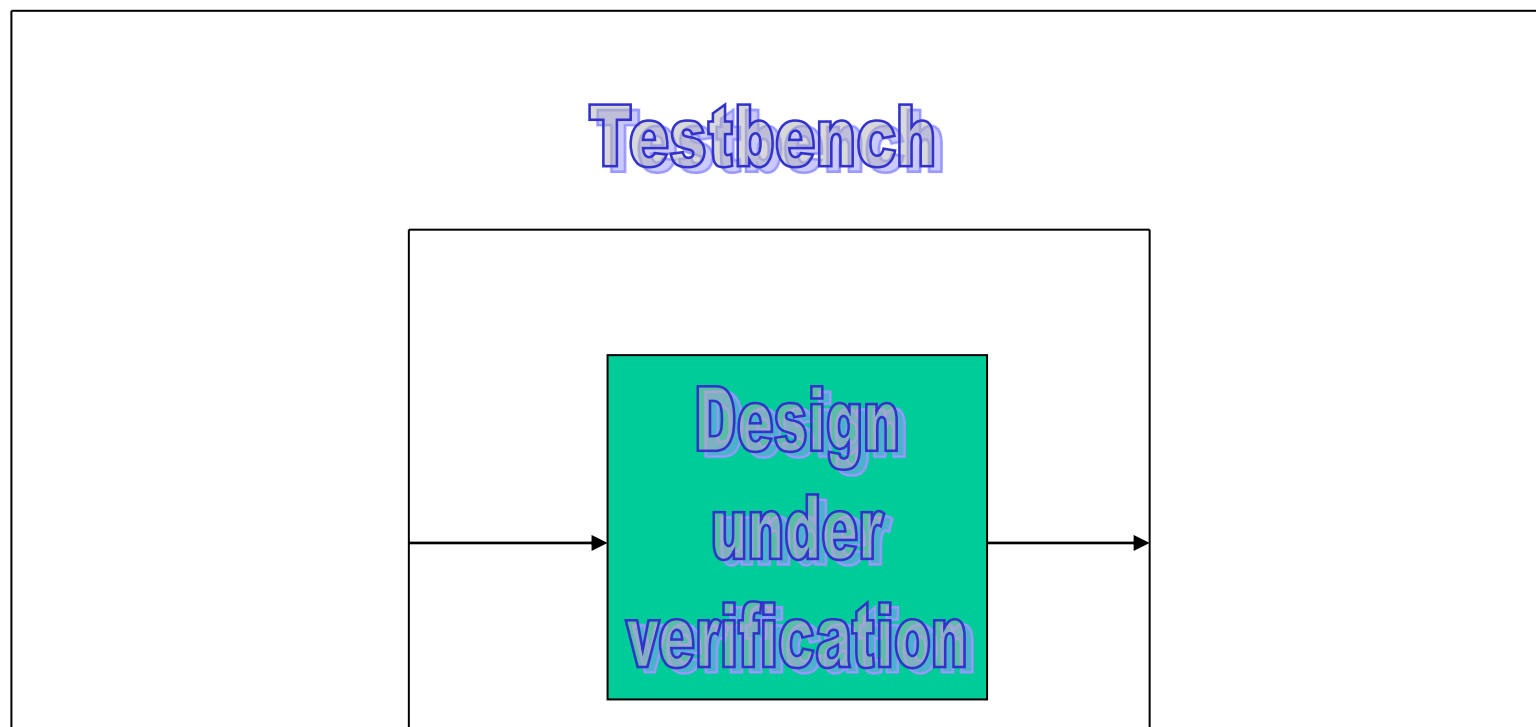


אימות יחידת החומרה (דוגמא)

```
if (RSTn = '0') then
    COUNT_VALUE <= 0;
elsif (CLK'event and CLK = '1')
then
    if (PL = '1') then
        COUNT_VALUE <= DATA;
    elsif (EN = '1') then
        if (COUNT_VALUE = 31)
        then
            COUNT_VALUE <= 0;
        else
            COUNT_VALUE <=
                COUNT_VALUE + 1;
        end if;
    end if;
end if;
```



סביבת בדיקות



איתותים, זמן וסימולציה

- **משתנים לעומת אותות**

- משתנים VHDL משנים ערך ללא דחיה (delay)

- לאותות VHDL יש דחיה

- לאות ניתן ערך בנקודה מסוימת בזמן, ואות שומרת על הערך עד שלו נתון ערך חדש

- waveform הוא רצף של הערכים לאורך הזמן

- **סימולטור האירועים מבצע קוד VHDL על ידי קידום זמן**
עד האירוע הבא, מעדכן ערכים של אותות ומתזמן אירועים חדשים

VHDL

Text Editor – verilog.v

File Edit Format Options

Help

```
`resetall
`celldefine
`delay_mode_path
`timescale 1ns/10ps
module inverter (out, inp);
    output out;
    input inp;
    not(out,inp);
endmodule
`endcelldefine
```

• חייב למדל לשער:

פונקציה ➤

עיכוב ➤

■ קיימים יחידות נתונות:

NAND, NOR, AND, OR, XOR, XNOR, BUF, ➤

NOT, and some others

הנחות -- Assumptions

ההנחה : לקחת משהו כנכון ללא הוכחה פורמלית .

הנחות בתכנון מעגלים משמשים לפשט את הניתוח או את התכנון .

המטרה של ההנחה היא להפריד בין מידע חשוב מן המידע הזניח.

האלמנטים של הנחה הם:

1. ניסוח ההנחה כדי לפשט את הבעיה ללא ביטול מידע חיוני.
2. יישום ההנחה לקבלת פתרון או תוצאה.
3. בדיקה כי ההנחה אכן מתאימה.

השמות -- Assignments

- דברים שחייבים להיות נכונים לגבי התכנון
- המשמר נגד תרחישים מסוכנים
- ניתן שצוין על ידי מהנדס אלקטרוניקה במסגרת המודל, או על ידי מהנדס בדיקות כחלק אימות
- ניתן להשתמש כדי לאמת את ההנחות שנעשו על ידי מהנדסים אחרים
- בודקות את הפונקציונאליות
- בודקות כי בסופו של התרחיש הנדרש קורה
- יכולות לשמש להוכחת נכונות התכנון



השמות ב-VHDL

```
module adder_4_RTL (a, b, c_in, sum, c_out);  
    output [3:0] sum;  
    output c_out;  
    input [3:0] a, b;  
    input c_in;  
  
    assign {c_out, sum} = a + b + c_in;  
  
endmodule
```

טענות - Assertion

```
assert boolean-expression  
    [report string-expression]  
    [severity expression];
```

The **assert** expression is an expression of a STRING type to be reported. The **severity** expression is a value of type SEVERITY_LEVE (a pre-defined enumerated type with four values **NOTE**, **WARNING**, **ERROR** and **FAILURE**).

דוגמא לשימוש ב-assertion

```
entity ASRTSTMT is
end ASRTSTMT;
architecture BEH of ASRTSTMT is
    constant SETUP : time := 3 ns;
    constant HOLD  : time := 3 ns;
    signal  D, CLOCK, Q : bit;
begin
    setup_check : process (CLOCK)
    begin
        assert FALSE report "Event on CLOCK" severity NOTE;
        if (CLOCK'event and CLOCK = '1') then
            assert D'STABLE(SETUP)
            report "D setup error" severity WARNING;
            if (D'STABLE(SETUP)) then
                Q <= D;
            end if;
        end if;
    end process;
end process;
```

Report statement

```
report string-expression  
    [severity expression];
```

The **report** statement is used to display a message. It is similar to an assertion statement, but **without the assertion check**.

דוגמא לשימוש ב-report

Example:

```
if CLR = 'Z' then
    report "Signal CLR has a high-impedance value."
    -- Default severity level is NOTE.
end if;

if CLK /= '0' and CLK /= '1' then
    report "CLK is neither a '0' not a '1'!!!"
    severity ERROR;
end if;
```

למה טענות?

- טענות מבטאות ידע על איך התכנון צריך להתנהג
- טענות משומשות בטכניקות מבוססות כיסויים וגם בטכניקות אימות בעזרת סימולציה, וגם אימות פורמליות
- טענות אמורות לסייע להגביר שקיפות התכנון, כמו גם בקרה על תהליך התכנון

למה טענות? (המשך)

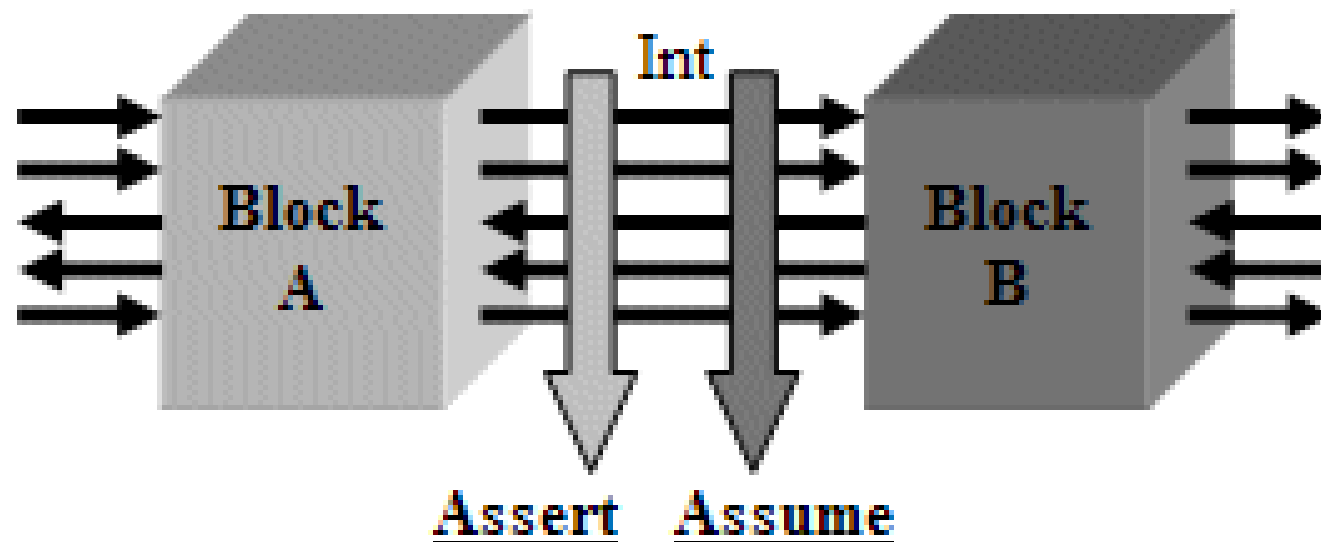
• כל טענה מציינת

- התנהגות חוקית של חלק כלשהו של הרכיב, או
- התנהגות בלתי חוקית של חלק מהרכיב

■ דוגמאות של טענות

- התור לא צריך לגלוש
- יש קבוצה של אותות שצריכה להיות "one-hot"
- אם האות עולה, אזי... ..

קשר בין הנחות וטענות



Int spacing ≥ 5 clocks

Int spacing ≥ 5 clocks

